

Equipping an AI Strength-and-Conditioning Coach: The Complete Science, Math, and Architecture

TL;DR

- **Build a deterministic “coaching engine” in code that does ~95% of the work** — e1RM estimation, RPE/%1RM conversion, volume tracking against MEV/MAV/MRV, fatigue modeling, and next-workout prescription are all closed-form math that should never touch an LLM; reserve the LLM purely for natural-language explanation, motivation, and genuinely ambiguous judgment calls.
- **The whole coaching domain reduces to a small set of well-established equations and lookup tables** — Prilepin’s chart, the seven e1RM formulas (Epley, Brzycki, etc.), the Tuchscherer/Zourdos RPE table, load-velocity profiling, the Banister fitness-fatigue model, and EWMA-based acute:chronic workload ratios — all of which are given below with exact numbers.
- **Handle cold-start with population-prior defaults and a named starter program (StrongLifts 5×5, GZCLP, or 5/3/1); handle data-rich users with autoregulation** driven by e1RM trends, velocity loss, and RPE feedback. This dual-mode design plus aggressive caching/precomputation keeps per-user LLM cost to a few cents.

Key Findings

1. **Almost everything a coach decides is computable.** Load selection, progression, volume management, and fatigue monitoring are governed by deterministic formulas. A non-coder can implement these as plain functions in Node.js. This is the single most important architectural insight: it is both better (deterministic, testable, free) and cheaper than asking an LLM to do arithmetic.
2. **e1RM is the central state variable.** Nearly every downstream calculation (percentage loads, RPE targets, volume-load normalization, plateau detection) keys off an estimated one-rep max. Use Epley or Brzycki for 1–10 reps; accuracy degrades sharply above ~10 reps.
3. **Two scientific debates must be surfaced honestly to the user:** (a) optimal training volume is genuinely individual and the MEV/MAV/MRV landmarks are heuristic ranges, not measured constants; (b) the “effective reps” model of hypertrophy is contested. A

good coach hedges these rather than asserting false precision.

4. **The LLM's irreplaceable jobs are linguistic and interpretive**, not computational: explaining *why* the plan changed, motivating adherence, parsing free-text user notes ("my shoulder felt tweaky"), and handling edge cases the rule engine doesn't cover.
5. **Token cost is controlled by architecture, not prompt golf**. Precompute in code, summarize history into a compact state object instead of sending raw logs, cache the static system prompt/knowledge, use structured outputs, and route mechanical sub-tasks to a small model.

Details

PART 1 — Coaching Science Across Training Styles (with the math)

Strength / Powerlifting

Maximal-strength work is organized around **intensity as %1RM**. The canonical dose-response guide is **Prilepin's Chart**, derived by A.S. Prilepin from the training logs of over 1,000 elite Soviet weightlifters:

% 1RM	Reps per set	Optimal total reps	Total range
<70%	3–6	24	18–30
70–80%	3–6	18	12–24
80–90%	2–4	15	10–20
>90%	1–2	7	4–10

The chart encodes a tradeoff: power output and technique degrade if you do too many reps at a given intensity, but too few reps under-stimulate. Common schemes fit inside it: 5×5 @ 75% = 25 reps (slightly above the strength sweet spot, good for size+strength), 6×3 @ 85% = 18 reps, 8×2 @ 90% = 16 reps. Rest periods for maximal strength are typically 3–5 minutes. Note: Prilepin's chart was built on the Olympic lifts and is applied successfully to powerlifting, but is rarely used in bodybuilding.

A refinement called **INOL** (Intensity × Number Of Lifts) exists for summing fatigue across multiple intensity zones, since naively adding each zone's optimal total reps grossly overestimates session volume.

Hypertrophy (muscle growth)

The primary driver is **mechanical tension** experienced by muscle fibers, with **proximity to failure** as the key controllable variable: as you approach failure, high-threshold motor units are progressively recruited, exposing type II fibers (greater growth potential) to high tension. McMaster University research established that muscle protein synthesis is driven primarily by mechanical load and proximity to failure, not metabolic stress markers like lactate.

Volume is programmed via the **Renaissance Periodization volume landmarks** (Dr. Mike Israetel & Dr. James Hoffmann), expressed as **direct sets per muscle group per week**:

- **MV (Maintenance Volume)**: ~6 sets/week maintains muscle. RP Strength
- **MEV (Minimum Effective Volume)**: fewest sets to grow; ~4–8 sets/week for many muscles in intermediates.
- **MAV (Maximum Adaptive Volume)**: the productive “sweet spot,” typically ~12–20 sets/week.
- **MRV (Maximum Recoverable Volume)**: the ceiling, ~18–30 sets/week depending on muscle and individual.

Programming logic: start a mesocycle at MEV, add sets weekly progressing toward MRV, then deload to MV and repeat. **Honest caveat**: these are heuristic ranges popularized by RP, not precisely measured constants; they vary by muscle, training age, and genetics, and the “effective reps” rationale behind them is contested — Stronger By Science argues the evidence for a discrete “last 5 reps matter most” model is weak. For most working sets, **RIR 0–3** is the productive hypertrophy zone (beginners RIR 3–4, advanced RIR 0–2). The most rigorous recent synthesis — Robinson, Pelland, Remmert, Refalo, Jukic, Steele & Zourdos (2024), “Exploring the Dose–Response Relationship Between Estimated Resistance Training Proximity to Failure, Strength Gain, and Muscle Hypertrophy: A Series of Meta-Regressions,” *Sports Medicine* 54(9):2209–2231 (55 hypertrophy + 67 strength studies) — found that hypertrophy increased as sets were taken closer to failure (a negative RIR slope) while “strength can improve at a wide range of RIR,” with the authors noting limitations in how RIR was estimated across studies.

Power / Explosiveness

Governed by the **force-velocity curve** and **rate of force development (RFD)**. Trained with **velocity-based training (VBT)** zones (see VBT math in Part 2). Methods include **contrast/complex training** (a heavy lift followed by an explosive one to exploit post-activation potentiation) and **dynamic-effort work** (submaximal loads moved maximally fast, e.g. Westside speed days). Typical bar-velocity zones (mean concentric velocity):

- Absolute strength: ~0.15–0.5 m/s
- Strength-speed: ~0.5–0.75 m/s
- Speed-strength / power: ~0.75–1.0 m/s
- Speed: ~1.0–1.3+ m/s

Muscular endurance & conditioning

Higher reps (15–30+), shorter rest (<60 s), and **density** (work per unit time) as the progression variable. Programmed by **work:rest ratios** — e.g. 1:1 for glycolytic conditioning, 1:3+ for power-endurance.

Cardiovascular / endurance

Intensity is anchored to heart rate zones. The **Karvonen formula** uses heart-rate reserve (HRR) for individualization:

Target HR = RestingHR + (MaxHR – RestingHR) × intensity% where HRR = MaxHR – RestingHR.

For estimating max HR, the **Tanaka formula (HRmax = 208 – 0.7 × age)** is the stronger default. Per Tanaka, Monahan & Seals, “Age-predicted maximal heart rate revisited,” *JACC* 2001;37(1):153–156, the equation was derived from “a meta-analytic approach...to collect group mean HRmax values from 351 studies involving 492 groups and 18,712 subjects,” then cross-validated in 514 subjects, with “HRmax was strongly related to age ($r = -0.90$)” — making it more accurate than the old “220 – age” (Fox) formula. Gulati (206 – 0.88 × age) is better for women. Five-zone model: Z1 50–60%, Z2 60–70% (aerobic base/fat oxidation, “conversational,” lactate <2 mmol/L), Z3 70–80% (tempo), Z4 80–90% (lactate threshold), Z5 90–100% (VO₂max). **Polarized training** (~80% easy Z1–2, ~20% hard Z4–5, little Z3) is favored over **threshold-focused** models for many endurance athletes, though both work.

Olympic weightlifting, CrossFit, calisthenics, kettlebell, sport-specific

- **Olympic WL:** low reps (1–3), high intensity, technical; Prilepin’s chart originates here.
- **CrossFit:** mixed-modal, time-priority or task-priority workouts; conditioning + barbell.
- **Calisthenics:** progression via leverage/ROM and added reps rather than external load; uses progressions (e.g. incline→full→archer→one-arm push-up).
- **Kettlebell:** ballistic (swing, snatch) work blends strength-endurance and power; density-driven.

- **Sport-specific/athletic:** concurrent training (strength + power + conditioning), often block- or conjugate-periodized.

Detecting a user's "lean"

Infer style from logged data: rep ranges (1–5 → strength; 6–12 → hypertrophy; 15+ → endurance), exercise selection (competition lifts → PL/WL; machines+isolation → bodybuilding; HR-tracked cardio → endurance), session structure, and stated goals. The coach should adapt the *engine's defaults* (volume landmarks, intensity anchors, progression rule) to the detected modality.

PART 2 — Periodization & Progression Mathematics

Periodization models

- **Linear (LP):** intensity ↑, volume ↓ over a mesocycle. Best for novices (e.g. add weight every session). Example weekly wave: W1 5×5 @75%, building load weekly.
- **Undulating (DUP/WUP):** vary intensity/volume frequently. Daily example: Power day 5×1 @90%, Strength day 4×5 @80–90%, Hypertrophy day 3×8–10 @60–80%. Superior fatigue management; suited to intermediates+.
- **Block:** sequential focused blocks (accumulation/hypertrophy → transmutation/strength → realization/peak), maintaining other qualities.
- **Conjugate / Westside:** concurrent max-effort (rotated every 1–3 weeks), dynamic-effort (speed), and repetition (hypertrophy) methods in one week. Note the semantic debate: "conjugate" technically refers to the rotation of max-effort exercises; the overall Westside system is essentially a specialized form of daily undulation.

Progressive overload quantification

- **Novice linear:** add a fixed increment each session — StrongLifts adds 2.5 kg/5 lb (2.5 lb on small lifts/women); GZCLP adds 10 lb lower / 5 lb upper per session.
- **Autoregulation:** adjust the *next* load from the *last* performance, using RPE or velocity rather than a fixed schedule.

RPE / RIR and the conversion math

RIR = 10 – RPE. RPE 10 = 0 reps left (failure), RPE 9 = 1 RIR, RPE 8 = 2 RIR, RPE 7 = 3 RIR, RPE 6 = 4 RIR.

The standard **Tuchscherer / Reactive Training Systems chart** (consistent with Zourdos et al. 2016) maps RPE × reps → %1RM. Full grid (% of 1RM; rows = reps, columns = RPE):

Reps	RPE 10	RPE 9.5	RPE 9	RPE 8.5	RPE 8	RPE 7.5	RPE 7	RPE 6.5	RPE 6
1	100%	97.8%	95.5%	93.9%	92.2%	90.7%	89.2%	87.8%	86.3%
2	95.5%	93.9%	92.2%	90.7%	89.2%	87.8%	86.3%	85.0%	83.7%
3	92.2%	90.7%	89.2%	87.8%	86.3%	85.0%	83.7%	82.4%	81.1%
4	89.2%	87.8%	86.3%	85.0%	83.7%	82.4%	81.1%	79.9%	78.6%
5	86.3%	85.0%	83.7%	82.4%	81.1%	79.9%	78.6%	77.4%	76.2%
6	83.7%	82.4%	81.1%	79.9%	78.6%	77.4%	76.2%	75.1%	73.9%
7	81.1%	79.9%	78.6%	77.4%	76.2%	75.1%	73.9%	72.8%	71.7%
8	78.6%	77.4%	76.2%	75.1%	73.9%	72.8%	71.7%	70.7%	69.6%
9	76.2%	75.1%	73.9%	72.8%	71.7%	70.7%	69.6%	68.6%	67.6%
10	73.9%	72.8%	71.7%	70.7%	69.6%	68.6%	67.6%	66.6%	65.6%

Read it both ways: to *prescribe*, look up the % for your target reps×RPE and multiply by 1RM; to *estimate 1RM* from a logged set, divide the load by the cell's %. Accuracy is ±3–5% for trained lifters; [Fitnesscalcs](#) values are barbell-back-squat means and shift for machine/single-joint/upper-body lifts. A quick closed-form alternative: with effective reps $ER = \text{reps} + RIR$, $\%1RM \approx 1 / (1 + ER/30)$ (the inverse of Epley).

Estimated 1-rep-max (e1RM) formulas

With W = weight lifted, R = reps completed, e = 2.71828:

- **Epley:** $1RM = W \times (1 + 0.0333 \cdot R)$ — equivalently $W \times (1 + R/30)$
- **Brzycki:** $1RM = W \times 36 / (37 - R)$ — equivalently $W / (1.0278 - 0.0278 \cdot R)$
- **Lander:** $1RM = (100 \times W) / (101.3 - 2.67123 \cdot R)$
- **Lombardi:** $1RM = W \times R^{0.10}$
- **Mayhew et al.:** $1RM = (100 \times W) / (52.2 + 41.9 \cdot e^{(-0.055 \cdot R)})$
- **O'Conner et al.:** $1RM = W \times (1 + 0.025 \cdot R)$

- **Wathan:** $1RM = (100 \times W) / (48.8 + 53.8 \cdot e^{(-0.075 \cdot R)})$ ShapeSense

Accuracy guidance: Epley and Brzycki are the most widely used and most accurate for ~2–10 reps; Brzycki is slightly more conservative (useful for picking a training max). Mayhew and Wathan use exponential curves and do better at higher reps (10–20). Lombardi tends to overestimate; O’Conner is most conservative. The LeSuer et al. (1997) study found Wathan, Epley, and Mayhew performed best across bench/squat/deadlift. **All formulas diverge badly above ~10 reps** — variance in muscular endurance between individuals makes high-rep estimates unreliable. Strength Journeys Practical rule: use sets of 3–6 reps, and average several formulas. Worked example (Epley, 100 kg × 5): $100 \times (1 + 5/30) = 116.7$ kg; Brzycki gives 112.5 kg. Arvo

Velocity-Based Training (VBT)

Because mean concentric velocity is near-perfectly inversely correlated with %1RM, you can build an **individual load-velocity profile (LVP)**: perform an incremental test at ~3–6 submaximal loads (ideally spanning 40–95% 1RM, at least one >80%), record the fastest rep’s mean velocity at each load, and fit a **linear regression** of velocity vs. load.

- **Minimum Velocity Threshold (MVT):** the slowest velocity at which a true 1RM can still be completed. The load where your LVP line crosses the MVT **is your estimated 1RM**. Population MVTs: ~0.30 m/s back squat, ~0.20 m/s bench, ~0.20 m/s deadlift (González-Badillo reports ~0.3 m/s squat, ~0.15 m/s bench). Individual measured MVT is more accurate.
- **Two-point method:** measure velocity at two distant loads (e.g. 45% and 85%), draw the line, read the 1RM at MVT — validated and low-fatigue.
- **Daily load picker:** the same line reads forward — pick today’s target velocity, read off the load; this autoregulates load to daily readiness.
- **Velocity loss (VL) as a fatigue cutoff:** stop the set when bar speed drops a set % from the fastest rep. Pareja-Blanco et al. (2017), *Scand J Med Sci Sports* 27:724–735 — an 8-week squat study (VL20 n=12 vs VL40 n=10) — found “VL20 resulted in similar squat strength gains than VL40 and greater improvements in CMJ (9.5% vs 3.5%, $P < 0.05$), despite VL20 performing 40% fewer repetitions,” whereas VL40 led to muscle failure in 56% of sets and maximized hypertrophy but caused a fast-to-slow fiber-type shift. Caveat: LVPs vary between individuals, exercises, and devices, and limits of agreement on 1RM estimates can be $\sim \pm 15$ kg on the squat.

Fatigue & recovery modeling — the Banister Fitness-Fatigue model

Performance is modeled as baseline plus a positive (fitness) impulse-response minus a

negative (fatigue) impulse-response to training load w :

$$P(t) = P_0 + \int_0^t w(t') \cdot g(t - t') dt', \text{ with transfer function } g(t) = k_1 \cdot e^{-(t/\tau_1)} - k_2 \cdot e^{-(t/\tau_2)}$$

arxiv

In discrete (daily) form:

- **Fitness:** $F(n) = \sum k_1 \cdot w(i) \cdot e^{-(n-i)/\tau_1}$
- **Fatigue:** $f(n) = \sum k_2 \cdot w(i) \cdot e^{-(n-i)/\tau_2}$ arxiv
- **Performance = P_0 + Fitness – Fatigue**

where k_1 , k_2 are the magnitudes of fitness/fatigue gain per unit load, and τ_1 , τ_2 are decay time constants. arxiv Fitness decays slowly, fatigue quickly (illustrative values from the literature: $\tau_1 \approx 20$ –42 days for fitness, $\tau_2 \approx 7$ days for fatigue; $k_2 > k_1$). Because fatigue is larger but shorter-lived, reducing load (a **taper/deload**) lets fatigue fall faster than fitness, raising performance — and a variable load program peaks higher than a constant one for the same total work. Modern platforms (TrainingPeaks) implement a simplified EWMA version: $g(t) = g(t-1) \cdot e^{(-1/\tau_1)} + w(t) \cdot (1 - e^{(-1/\tau_1)})$, branded as CTL (Chronic Training Load = fitness), ATL (Acute Training Load = fatigue), and TSB (Training Stress Balance = form = fitness – fatigue).

Training load quantification — TRIMP & TSS

- **Banister TRIMP** = duration × HRR-fraction × weighting factor, uspto where HRR-fraction = $(\text{avgHR} - \text{restHR}) / (\text{maxHR} - \text{restHR})$, and the weighting factor is exponential: $0.64 \cdot e^{(1.92 \cdot x)}$ for men, $0.86 \cdot e^{(1.67 \cdot x)}$ for women (x = HRR fraction). Worked example: male, HRmax 200, HRrest 40, 30 min @ 130 bpm → $x = 0.5625$ → TRIMP ≈ 32 .
- **TSS (Coggan):** $\text{TSS} = (\text{duration}_s \times \text{NP} \times \text{IF}) / (\text{FTP} \times 3600) \times 100$, where $\text{IF} = \text{NP}/\text{FTP}$. One hour at threshold = 100 TSS by definition.
- **Session RPE (Foster):** $\text{sRPE load} = \text{session_RPE}(1-10) \times \text{duration_minutes}$. Simplest and validated; ideal for a gym app that lacks HR/power data.

Acute:Chronic Workload Ratio (ACWR)

ACWR = acute load (≈ 7 -day) / chronic load (≈ 28 -day). Two methods:

- **Rolling average (coupled):** acute / mean(28-day).
- **EWMA (preferred):** $\text{EWMA}_{\text{today}} = \text{Load}_{\text{today}} \times \lambda + (1 - \lambda) \times \text{EWMA}_{\text{yesterday}}$, where $\lambda = 2/(N + 1)$ (N = the time-decay window, e.g. 7 for acute, 28 for chronic). EWMA weights recent load more and was shown (Williams et al.) to be more sensitive to injury

risk than rolling averages.

- **Interpretation:** a “sweet spot” of ~0.8–1.3 is associated with lower injury risk; **>1.5 (especially >2.0) signals a load spike and elevated injury risk**, <0.8 suggests detraining. **Honest caveat:** the ACWR-injury link has been challenged on methodological grounds in recent literature, so treat it as a monitoring flag, not gospel.

Volume-load & normalization

- **Volume load (tonnage)** = Σ (sets $\times$ reps $\times$ weight). The basic workload currency.
- **Relative intensity** = working weight / e1RM (or average %1RM across sets).
- **Normalizing across exercises:** since absolute tonnage isn’t comparable between, say, deadlift and curl, normalize by expressing each exercise’s load relative to its own e1RM, or track **hard sets per muscle per week** (the RP currency) rather than raw tonnage for hypertrophy.

Deload logic

Heuristics that should be coded as rules:

- **Scheduled:** every 4th week (5/3/1) or at end of a mesocycle when volume has progressed MEV→MRV.
- **Autoregulated triggers (deload when ≥ 2 fire):** e1RM or rep-PR performance drops for 2+ consecutive sessions despite effort; persistent soreness not resolving; rising resting HR / poor sleep; motivation drop; ACWR spiking >1.5; bar velocity at a fixed load down ~10% vs. recent baseline.
- **Execution:** drop to ~40–60% of training max / MV sets for a week (5/3/1 week-4: 40% $\times$ 5, 50% $\times$ 5, 60% $\times$ 5), then resume.

Named program math (for the engine’s “templates”)

- **StrongLifts 5 $\times$ 5:** Workout A (Squat 5 $\times$ 5, Bench 5 $\times$ 5, Row 5 $\times$ 5) / B (Squat 5 $\times$ 5, OHP 5 $\times$ 5, Deadlift 1 $\times$ 5), 3 $\times$ /week alternating; +2.5 kg/5 lb per session when all reps hit; fail → retry, then deload 10%; deload by switching 5 $\times$ 5→5 $\times$ 3→5 $\times$ 1.
- **GZCLP:** T1 main lift 5 $\times$ 3+ (last set AMRAP), +10 lb lower/+5 lb upper per session; on fail, 5 $\times$ 3→6 $\times$ 2→10 $\times$ 1→reset. T2 3 $\times$ 10 moderate; T3 3 $\times$ 15+ isolation. Each main lift trained twice/week.
- **5/3/1 (Wendler):** all percentages off a **Training Max = 90% of true 1RM** (Wendler: “start too light”). 4-week cycle:

- W1: 65%×5, 75%×5, 85%×5+ (AMRAP)
 - W2: 70%×3, 80%×3, 90%×3+
 - W3: 75%×5, 85%×3, 95%×1+
 - W4 (deload): 40%×5, 50%×5, 60%×5
 - Add +5 lb upper / +10 lb lower to the TM each cycle; if the AMRAP 1+ set drops below 3 reps, reset TM by 10%. Estimate the next TM from the AMRAP via Epley.
-

PART 3 — Feeding Intelligence Into an AI Coach

Knowledge representation: code vs. LLM parametric knowledge

The reliable pattern is **structured/symbolic knowledge for anything deterministic**, LLM only for fuzzy language. Concretely:

- **Lookup tables & rules in code/JSON:** Prilepin's chart, RPE table, volume landmarks per muscle, program templates, deload triggers, e1RM formulas. These are exact, testable, and free. Structured prompting (inlining a small JSON of rules) is a simpler, more deterministic alternative to RAG when the knowledge domain is "well-defined and stable" — which coaching largely is.
- **LLM parametric knowledge** should *not* be relied on for numbers (it will hallucinate a percentage or formula). Use it for phrasing and interpretation.

Retrieval-Augmented Generation (RAG): when and how

RAG matters when you have *unstructured* knowledge (e.g. a library of coaching articles, exercise technique cues, injury-modification notes) the LLM should quote. Pipeline: chunk documents (semantic chunking preserves whole concepts better than fixed-size), embed chunks into a vector store, embed the query, retrieve top-k ($k \approx 3$), optionally compress, then stuff into the prompt with a "use the information below" instruction. **Key insight from a real fitness-app builder:** for *structured records* (the user's workout log), RAG is the wrong tool — "the data is structured records, not documents." Query SQL/Sheets directly. Reserve vector search for the prose knowledge base, and consider it a later-stage feature.

Separating deterministic logic from the LLM

This is the heart of a cheap, reliable coach. Allocate responsibilities:

- **Code (deterministic engine):** all math (e1RM, %1RM, velocity profiles, TRIMP/ACWR,

volume tallies), all progression decisions (what weight/reps next), all safety rules (deload triggers, load-spike caps), program template state machines.

- **LLM:** natural-language explanation of the engine's decision, motivation, parsing free-text logs/notes into structured data, answering open user questions, and flagging genuinely novel situations. As the Fitness Bridge developer put it: "If the coaching advice feels off, I know to look at the prompt, not the math" Medium — because the math lives in code.

Training history as data: feature engineering

From the logged sets (your Sheets layer), compute a compact **feature/state object** per user per lift:

- **e1RM trend** (slope over last N sessions; flag plateau if slope ≈ 0 or negative over ~3–4 weeks).
- **Volume trend** (weekly hard sets per muscle vs. landmarks).
- **Fatigue indicators** (ACWR/EWMA of sRPE load; velocity drop; RPE creep at fixed load).
- **Adherence** (sessions completed / planned).
- **PRs & staleness** (sessions since last rep-PR or e1RM-PR). This object — not raw logs — is what gets summarized to the coach.

Cold-start vs. data-rich

- **No history (cold start):** a short questionnaire (training age, goal, days/week, equipment, injuries, est. or tested maxes) → population-prior defaults → assign a standard starter program. Novice → StrongLifts 5×5 or GZCLP (linear, fast feedback); some intermediates → 5/3/1. Use conservative TM (90% rule). This is pure decision-tree logic; no LLM needed.
- **Rich history:** switch to autoregulation — set loads from current e1RM/velocity, adjust volume against landmarks, trigger deloads from fatigue features, and personalize the RPE chart per lift (RTS recommends individualized charts because per-lift rep-max curves differ).

"Training the AI": fine-tune vs. prompt vs. RAG vs. tools

For a solo project, the ROI order is clear:

1. **Tool-use / function-calling (highest ROI):** let the LLM call your deterministic functions (`getNextWorkout`, `estimate1RM`, `getVolumeStatus`). This converts natural-language

requests into reliable structured calls and keeps all math in code.

2. **Prompt engineering + structured outputs:** a compact system prompt encoding coaching tone and the small rule set; request JSON.
 3. **RAG:** only once you have a real prose knowledge base worth retrieving.
 4. **Fine-tuning (lowest ROI for a solo dev):** expensive, data-hungry, and unnecessary when tools+prompting already give correct behavior. Skip it.
-

PART 4 — Token Efficiency & LLM Cost Optimization

The user's explicit concern — "make as much of it possible without the LLM to save on tokens" — is best answered by architecture. Pricing is linear: $\text{cost} = \text{input_tokens} \times \text{in_price} + \text{output_tokens} \times \text{out_price}$, and **output tokens cost roughly 5x input tokens** on current frontier models (e.g. Claude Sonnet 4.6 at \$3/M input vs \$15/M output; Haiku 4.5 at \$1/\$5; Opus 4.6 at \$5/\$25 — a 1:5 ratio; some practitioner guides cite 4–8x), so constraining output has outsized impact.

When NOT to call the LLM at all (pure code paths):

- Computing the next workout's loads/reps/sets.
- Any e1RM, %1RM, velocity, volume, TRIMP, ACWR, or deload calculation.
- Rendering the workout, plate math, rest timers, charts.
- Progression decisions and program template advancement. These are 100% deterministic — calling an LLM for them is both a hallucination risk and pure waste.

Strategies (stacked, these cut cost 60–85%):

- **Precompute with code, then optionally narrate.** Compute the workout in Node; pass the LLM only a tiny structured summary if the user wants an explanation.
- **Summarize history, never send raw logs.** Send the compact state object (e1RM trend, volume status, fatigue flags) — not hundreds of logged sets. One real builder keeps the last ~8 messages verbatim and regenerates a compressed summary when history exceeds ~20 messages.
- **Prompt caching.** Cache the static system prompt + knowledge block. Per Anthropic's prompt-caching docs (introduced Aug 2024), cache reads are billed at 0.1x standard input — a **90% discount** — with cache writes at 1.25x for a 5-min TTL / 2x for a 1-hr TTL; OpenAI's automatic caching gives **50% off cached input** on prompts $\geq 1,024$ tokens with no code changes (newer GPT-5.x models have moved to ~90% off cached

input). With repeated structure this is the single biggest lever.

- **Structured outputs (JSON schema) + max_tokens.** Forces concise, parseable responses; eliminates rambling and rework. "Single, precise generation is always better than multiple iteration loops."
- **Model routing.** Use a small/cheap model (e.g. a Haiku-class model) for mechanical language tasks (parsing a note, formatting), reserve a larger model only for genuine reasoning. Reported 40–70% savings.
- **Semantic/exact caching of responses.** Cache common explanations ("why deload?") keyed by the decision type; serve cached prose for repeat situations. 50–90% reduction on repeats.
- **Batching** for non-urgent background jobs (e.g. weekly summaries) — ~50% discount on some providers.

Concrete coaching pattern: Engine computes "Squat: 102.5 kg × 5×5, +2.5 kg from last week because all reps hit at RPE ≤8." The app *displays* that directly (zero tokens). Only if the user taps "explain" or "I feel beat up" does the LLM fire — receiving the one-line state, not the log — and return two sentences. Most sessions cost **zero** LLM tokens.

PART 5 — Recommended Architecture (Blueprint)

Layers (maps onto your PWA + Sheets + Node stack):

1. **Data layer (Google Sheets):** raw set logs (date, exercise, weight, reps, RPE, optional velocity), user profile, program state. One row per set.
2. **Deterministic Coaching Engine (Node.js) — the core, does ~95% of work:**
 - *Math module:* e1RM (Epley/Brzycki + averaging), RPE↔%1RM table lookup, Karvonen, TRIMP/sRPE, EWMA/ACWR, volume-load.
 - *Feature module:* builds the per-user state object (trends, fatigue, PRs, adherence).
 - *Decision module:* program templates (5×5, GZCLP, 5/3/1) as state machines + autoregulation rules + deload triggers + Prilepin/landmark guardrails.
 - Output: a fully specified next workout + a compact rationale string.
3. **LLM layer (thin):** function-calling interface exposing the engine; small cached system prompt; structured JSON I/O; invoked only for explanation, motivation, free-text parsing, Q&A, and edge cases. Cheap model for mechanical tasks, capable model for reasoning. Optional RAG over a prose knowledge base later.

4. **Presentation (PWA):** renders engine output, charts e1RM/volume/fatigue trends, logs sets back to Sheets.

How it satisfies the four requirements:

- **(a) No-data users:** questionnaire → decision-tree → starter program with conservative defaults. No LLM math.
- **(b) Data-rich users:** autoregulation from e1RM/velocity/RPE features; per-lift personalization; fatigue-triggered deloads.
- **(c) Multi-style:** engine carries modality profiles (strength→Prilepin/%1RM; hypertrophy→landmarks/RIR; power→velocity zones; endurance→HR zones/polarized; conditioning→work:rest), selected by detected lean or user choice.
- **(d) Token-minimal:** all math/decisions in code; LLM optional and cached.

Authoritative sources/frameworks to encode: NSCA *Essentials of Strength Training and Conditioning* (foundational principles, VBT theory-to-application); **Prilepin's chart** (strength dose-response); **Renaissance Periodization** volume landmarks (Israetel/Hoffmann); **Helms et al. (2016, 2018)** and **Zourdos et al. (2016)** for RIR-based RPE; **Tuchschere/RTS** RPE chart; **Banister (1975)** fitness-fatigue and **Busso** variable-dose refinement; VBT literature from **Bryan Mann** and **González-Badillo/Sánchez-Medina**; **Coggan** (TSS) and **Foster** (sRPE) for load; **LeSuer et al. (1997)** for e1RM formula accuracy.

Recommendations

Stage 1 — Build the deterministic engine first (no LLM). Implement the math module (e1RM, RPE table, volume, Karvonen, EWMA/ACWR) and one starter program state machine (StrongLifts 5x5 or GZCLP). This alone is a functional coach. Benchmark: the app should prescribe every set with zero LLM calls.

Stage 2 — Add the feature/state object and autoregulation. Compute e1RM trends, volume vs. landmarks, fatigue flags; add deload triggers and 5/3/1 for graduating users. *Threshold to advance a user from LP to autoregulation:* when session-to-session linear progress stalls twice despite a deload (typical at ~squat 80–100 kg / bench 60–80 kg / deadlift 100–130 kg for men, proportionally less for women).

Stage 3 — Bolt on the thin LLM layer with function-calling + caching. Expose engine functions as tools; cache the system prompt; structured JSON; cheap model for parsing, capable model for explanations. *Threshold to consider RAG/fine-tuning:* only if you accumulate a real prose knowledge base or measure systematic explanation errors — otherwise skip both.

Stage 4 — Instrument cost and accuracy. Log tokens per interaction; if any calculation is being done by the LLM, move it to code. *Trigger to re-architect:* if average per-session token cost exceeds a few cents, a calculation has leaked into the LLM.

What would change these recommendations: If you later want VBT, add a velocity field and the LVP regression (high value for autoregulation). If users are pure endurance athletes, lead with HR-zone/TSS logic instead of %1RM. If volume-landmark guidance produces overtraining complaints, lower default MEV/MAV starting points (they're individual).

Caveats

- **Volume landmarks (MEV/MAV/MRV) are heuristics, not constants.** They vary by muscle, training age, and genetics; present them as starting ranges and adjust from the user's own performance/recovery data.
- **e1RM is unreliable above ~10 reps** and differs by formula and lift; use low-rep sets and average formulas, and treat e1RM as an estimate with $\pm 3\text{--}5\%$ error.
- **The RPE→%1RM chart is a population mean** (built on the back squat) with $\sim \pm 3\text{--}5\%$ error and meaningful individual variation; personalize per lift once you have data, and note novices rate RPE less accurately than trained lifters.
- **"Effective reps"/proximity-to-failure as the hypertrophy mechanism is scientifically contested** (Stronger By Science vs. RP); don't assert it as settled.
- **ACWR's injury-prediction validity has been challenged;** use it as a load-spike flag, not a hard safety guarantee.
- **VBT 1RM estimates carry wide limits of agreement** ($\sim \pm 15$ kg on the squat in some studies) and depend on device/exercise; individual MVT beats population values.
- **Fitness-fatigue model parameters (k_1 , k_2 , τ_1 , τ_2) must be fit per individual** and are noisy; the model is best used qualitatively (taper logic) unless you have enough data to fit it.
- **Heart-rate max formulas are estimates** ($\pm 10\text{--}12$ bpm typical); a tested max or lactate testing is more accurate.
- **None of this is medical advice;** the app should include injury/medical disclaimers and conservative defaults, especially for beginners and anyone with health conditions.